CHAPTER 07 DATA ENGINEERING / CONTINUOUS INGESTION

Snowpipe, Streaming, and Real-Time Ingestion

Build a pipeline whose freshness claims survive failures and replay

The alert was early. The evidence arrived late.
At 10:05 an energy operator receives a grid-balancing alert. The dashboard still shows 09:45 generation figures, although sensors have transmitted readings every second. The team asks whether Snowflake is "slow." But the missing time may be in a message broker, an object-storage batch, a cloud notification, a paused consumer, or the transformation after ingestion. The engineering question is not merely how rapidly bytes enter a table. It is *how soon a trustworthy answer becomes visible to its intended reader.*

Chapter 5 designed reproducible file loading; Chapter 6 taught us to preserve, inspect and validate evolving event payloads. We now ask when files are enough and when continuously arriving rows justify a new ingestion contract. The Finland Energy Analytics Platform will retain its bronze–silver boundary, immutable event identity and UTC observation grain, but introduce event clocks, delivery offsets, operational visibility and recovery tests.

After this chapter you can:

- choose scheduled `COPY INTO`, event-triggered Snowpipe or Snowpipe Streaming from measurable service needs;
- explain cloud notifications, stages, pipes, streaming channels and checkpoint responsibilities;
- separate source ordering, arrival ordering, revision precedence and query visibility;
- exercise duplicate delivery, late observations, schema drift and transient failure safely;
- present a defensible freshness and cost measurement plan without inventing cloud results.

Prerequisites: Chapters 3, 5 and 6; basic SQL and Python. The core lab is offline, with synthetic data. Live Snowflake examples are reviewed against vendor documentation but *not executed in a customer account.* Region, edition, SDK version and consumption pricing must be rechecked before deployment.



Figure 7.1. Freshness is an end-to-end property, not the ingestion API's name.

7.1 Batch versus near-real-time versus streaming

7.1.1 Throughput and latency definitions

A pipeline accepts units of work and exposes results. **Throughput** is the number of events, rows or bytes successfully processed per unit time; **latency** measures elapsed time for one particular work item. A system can exhibit excellent throughput while individual messages wait in a queue. Conversely, a tiny live demo can show low latency but collapse under an actual peak. Never infer one metric from the other.

Distinguish at least four clocks: *event time*, when the device says the measurement applies; *source publish time*, when the sender hands it to transport; *ingest visibility time*, when the accepted bronze record can be queried; and *business readiness time*, when validation and aggregation make it safe for the dashboard. A reliable SLA explicitly chooses the interval it measures, including its treatment of late source publication and failed arrivals.

Operational vocabulary
Event-to-visible age: $t_{\text{visible}} - t_{\text{event}}$. **Ingestion delay:** $t_{\text{visible}} - t_{\text{publish}}$. **Business freshness:** how old the most recent *accepted* observation is for a required key. **Backlog:** work accepted upstream but not yet committed and validated downstream.

Listing 1: Measure received-to-event age when both UTC timestamps are recorded.

```
SELECT technology,
  DATEDIFF('second', event_utc, received_utc)
    AS observed_source_age_seconds
FROM energy_telemetry_raw;
```

That query does *not* measure actual Snowflake ingest service latency unless the receive timestamp was recorded at a reliably defined visibility boundary. The lab uses synthetic times precisely to exercise the formula; it makes no vendor speed claim. In production, collect enough rows for percentile statistics and stratify by provider, technology and hour: one median can hide an unacceptably slow tail. **Decision check.** A stakeholder says "we need real time." Ask: Freshness of which metric? Measured from which clock? What proportion must meet the target? How long may corrected values lag? These four answers determine architecture more usefully than the phrase itself.

Project reference: P007: Snowflake time-series streaming guide. *Not independently executed; rights review pending.*

7.1.2 Batch, micro-batch, and continuous designs

Scheduled batch loads move source snapshots or files at fixed intervals. They are easy to inspect, replay and reconcile; they add waiting time between arrivals. Micro-batching reduces that wait by processing frequent small groups, but every batch still needs discovery, serialization, commit and monitoring. Row-oriented streaming accepts new events without first requiring an object-storage file. None of these modes automatically produces a correct analytical table.

Pattern	Strong fit	Primary burden
Scheduled COPY	Hourly/daily immutable exports	Scheduler; batch boundaries
Snowpipe files	Frequent cloud-object arrivals	Notification plumbing; file lifecycle
Streaming API	Continuous event feeds	Checkpoints, retries, channel health

Table 7.1. Select against workload requirements, not product branding.

Suppose a market report refreshes once an hour and source files arrive every fifteen minutes. Scheduled loading may be sufficient even if the source calls its service a "stream." If operators need a sub-minute view of sensor fault indicators, waiting for objects to be created and discovered may be inappropriate. If the source only publishes once per hour, Snowpipe Streaming cannot invent more recent observations.

A subtle architectural trap: event frequency and decision frequency are different. Thousands of sensor events per second may be aggregated into five-minute operational windows, yet incident detection may depend on an instantaneous threshold. Split the design when workloads have distinct freshness and quality demands; do not send every raw event through the most expensive path merely because some consumers need fast alerts.

Design against two SLAs.
The Finland energy example maintains *bronze ingest visibility* and *validated silver reporting readiness* as separate service objectives. Fast bronze ingestion with an hours-old silver watermark is still an operational failure for the analyst.

Check. Sketch two plans for a source producing one 2-MB object per minute: a scheduled load every fifteen minutes and event-triggered Snowpipe. Name every place a file can wait, and state which additional moving parts the second design introduces.

7.1.3 Costs, failure modes, and selection discipline

Cost has both a metered and an engineering component. Batch loading typically consumes virtual-warehouse credits while the scheduled load runs. File-based Snowpipe uses Snowflake-managed ingestion services, with billing rules that are not identical to a user-managed warehouse. High-performance Snowpipe Streaming has a throughput-oriented ingestion billing model; actual charges depend on current account pricing and the volume measured by the service. Downstream SQL transformations, retained storage, notification infrastructure and monitoring may add costs in every option.

A useful comparison fixes the *same accepted records* and the *same business freshness objective*. Otherwise a cheap hourly batch and a continuously validated streaming system are solving different problems. Record: expected events/day; peak-to-mean ratio; average payload and file size; maximum acceptable loss and duplicate rate; replay horizon; human on-call time; retention; network egress; and idle-resource billing.

Why fast ingestion can be expensive operationally
The fragile implementation fails when a client restarts, processes the same broker offset twice, mistakes "sent" for "committed," loses its schema version, or publishes a late correction without adjusting the aggregate. Recovery and observation are part of streaming architecture, not optional polish.

Listing 2: Design-time arithmetic; illustrative numbers, not vendor prices.

```
events_day = 86400 * 4          # four events per second
mean_bytes = 280                # application data estimate
estimated_GB_day = events_day * mean_bytes / 1e9
print(round(estimated_GB_day, 3))  # 0.097 GB/day
```

Even that number is not a bill: the accounting basis can exclude JSON structural bytes and include additional service classes. Avoid extrapolating a small simulator into a dollar forecast without the account's rate card.

Decision procedure. First confirm the source can deliver incremental data. Second define a p95/p99 freshness requirement and replay guarantee. Third run an identical input through two feasible patterns. Fourth measure downstream query visibility, data quality and metered usage. Finally document why the extra complexity is warranted. This chapter's controlled experiment provides the metrics framework; it does not claim to settle the choice for every team.

7.2 Snowpipe architecture and auto-ingest concepts

7.2.1 Files, stages, and the PIPE object

File-based Snowpipe is a managed service that uses a `PIPE` object to define a `COPY INTO` load from a stage. It loads new staged files without requiring you to schedule a virtual-warehouse `COPY` operation for each delivery. The pipe records loading instructions; storage integration and event publication are separate concerns. The pipeline still needs a stable file naming policy, predictable file formats and a contract for partial or rejected records.

Listing 3: File-based pipe definition; requires an approved external stage.

```
-- ENERGY_EVENTS_EXT and storage permissions are
-- provisioned separately. No cloud queue is created.
CREATE OR REPLACE PIPE ENERGY_FILE_PIPE
  AUTO_INGEST = FALSE AS
COPY INTO ENERGY_TELEMETRY_RAW
FROM @ENERGY_EVENTS_EXT
FILE_FORMAT = (TYPE = JSON);
```

7.2.2 From cloud event to a visible row

For auto-ingest, supported object-storage notifications announce new file objects; cloud-specific queue or subscription plumbing makes these announcements available to Snowflake. Snowpipe matches notifications to relevant staged paths and executes the pipe's load definition using managed compute. Visibility occurs later than a cloud's "object created" event; track both instants when diagnosing delay. An event is not the file itself, and a pipe declaration alone does not configure the notification service.

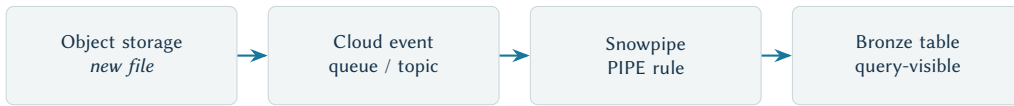


Figure 7.2. Events trigger file discovery; files hold the source records.

7.2.3 Arrival and load observability

`SYSTEM$PIPE_STATUS` exposes operational state for a pipe, including notification-related fields for auto-ingest where available. Load history and error notifications give different views: do not treat an empty error log as proof of complete source delivery. Reconcile file manifests against accepted rows and keep rejected-record provenance.

Project reference: P048: Auto-ingest demonstration. *Not independently executed; rights review pending.*

7.3 Cloud event notifications and staged files

7.3.1 Event topics, queues, and subscriptions

An object-store notification announces that a file has appeared or changed. Depending on cloud provider, configuration can involve S3 events and queues, Azure event infrastructure or Google Cloud Pub/Sub; Snowflake's integration instructions are cloud-specific. Specify an event subscription restricted to the intended path and event type. Broad notifications can waste work, expose sensitive object names or route unrelated files to the same ingestion path.

7.3.2 Storage integration as a trust boundary

A stage tells Snowflake *where* to look; storage integration determines which permitted identity may read it. Principle of least privilege applies to both the Snowflake role that creates or operates pipes and the cloud identity authorized to access objects. Prefer dedicated paths per environment, and avoid embedding cloud access keys in book code. Before deployment, check region constraints and organizational data-residency policy.

Common failure: notifications succeed, access fails.
Cloud monitoring shows the object-created event was published, yet no data appear. The pipe may lack rights to read the object, the stage may target a different prefix, the file format may reject records, or a queue permission may block delivery. Diagnose *object access*, *notification receipt*, *copy execution*, and *silver validation* as separate gates.

7.3.3 Duplicates, replays, and delivery assumptions

Cloud notifications may be repeated or delayed; object keys may be reused; producers may overwrite existing keys. A safe publication policy uses immutable object names that encode source partition and batch identity. Do not claim that a repeated notification always adds duplicate table rows; Snowflake tracks file loading in supported workflows, but its load metadata retention and reloading behavior have limits. Your source-level deduplication and reconciliation policy remains necessary.

Listing 4: Inspect a configured pipe without changing data.

```
SELECT SYSTEM$PIPE_STATUS('ENERGY_FILE_PIPE');
-- Inspect notification queue state and error fields
-- in context of this account's cloud provider.
```

Recovery drill. A cloud notification is lost while the source file remains in object storage. Demonstrate a bounded, audited backfill procedure based on a manifest and ingestion history, not a blind "force reload every file" script. Document who may approve a replay and which analytical keys are protected from double-counting.

Project reference: P056: Terraform Snowpipe cloud plumbing. *Not independently executed; rights review pending.*

7.4 Snowpipe Streaming fundamentals

7.4.1 Architecture: classic and high-performance paths

Snowpipe Streaming accepts event records directly through a supported ingest client rather than first discovering a staged file. As of this manuscript's documentation checkpoint, two architectural families matter. The *classic* approach uses its original SDK, channel conventions and operational limitations. The *high-performance* approach uses newer SDK/REST interfaces and a server-side `PIPE` to accept and, where configured, transform streamed rows. They are not merely two names for identical code. Follow the documentation for the client and architecture actually selected.

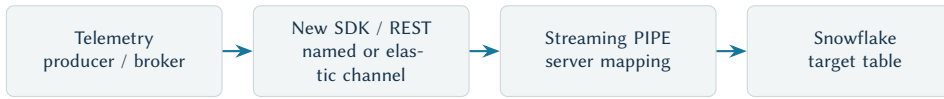


Figure 7.3. High-performance streaming uses a server-side pipe entry point.

In the high-performance architecture, you can define a named pipe for custom mapping, or use a supported default-pipe workflow. Where the architecture and SDK support them, *named channels* allow explicit offset tracking. Some interfaces also offer *elastic channels* with a different delivery model. Do not transfer an exactly-once claim from one channel mode to another. Those terms have consequences for retries and deduplication.

The database boundary matters. A successful SDK submission, a committed channel offset and a business-valid silver row are three distinct states. For the energy platform, raw telemetry carries event identity, source partition/offset, original event time, producer schema version and processing status. Transformations that compute publication-ready MWh totals occur downstream, where duplicates and revisions can be resolved with domain rules.

Project reference: P068: Snowpipe Streaming SDK examples. *Not independently executed; rights review pending.*

7.4.2 Rows versus files: the engineering boundary

File-based Snowpipe and row-based Snowpipe Streaming can both produce bronze records, but the delivery units differ. A file pipeline needs object naming, object lifecycle, cloud notification integration and file-load auditing. A row pipeline needs channel identity, source offsets, client resilience, schema mapping and row-error behavior. Shared downstream obligations include time semantics, privacy, reconciliation, data quality and cost attribution.

Dimension	Snowpipe files	Streaming API
Transport unit	Cloud object	Row / client batch
Primary identity	File path + load state	Source event and channel offset
Discovery	Notification or refresh	Client submission
Recovery	Manifest, load history, backfill	Checkpoint, replay, row errors
Typical source	Periodic exports	Broker, IoT, CDC event stream

Table 7.2. Both options need explicit correctness contracts.

Do not confuse Snowpipe Streaming with the SQL STREAM object introduced later in Chapter 8. The former is an *ingestion interface*; the latter tracks changes to a Snowflake table for downstream consumption. Likewise, receiving CDC messages from an upstream database is not the same as using a Snowflake change-tracking stream. Similar vocabulary hides different ownership and failure modes.

Why not stream everything?
An hourly regulator report that must be signed off against immutable batches benefits from file manifests and bounded reconciliation. A high-frequency sensor feed requiring rapid anomaly identification may prefer a streaming client, but if the business reprocesses daily for reporting, it can still retain an immutable archive for audit. Mixed paths can coexist when their acceptance and reconciliation rules are clear.

Check. Compare a source sending 10,000 tiny files per hour with a source sending 10,000 events per hour over a broker. Which ingestion unit is natural in each case? What changes in operational burden when you force one source into the other's delivery unit?

Project reference: P070: Official Snowflake Kafka connector. *Not independently executed; rights review pending.*

7.4.3 Supported interfaces and deployment checks

For the high-performance architecture, official Snowflake materials document a newer Java/Python SDK family and REST endpoints, with interface-specific behavior for channel operations. Availability, implementation limits, SDK APIs, region coverage and throughput ceilings are versioned product facts, not permanent design axioms. Freeze exact library versions and check the current documentation immediately before execution or publishing.

Listing 5: Illustrative high-performance streaming pipe syntax.

```
CREATE OR REPLACE PIPE ENERGY_EVENTS_STREAM_PIPE AS
COPY INTO ENERGY_TELEMETRY_RAW
  (EVENT_ID, TECHNOLOGY, EVENT_UTC,
   ENERGY_MWH, REVISION)
FROM (SELECT
  $1:event_id::VARCHAR,
  $1:technology::VARCHAR,
  TRY_TO_TIMESTAMP_NTZ($1:event_utc::VARCHAR),
  TRY_TO_DECIMAL($1:energy_mwh::VARCHAR,18,3),
  TRY_TO_NUMBER($1:revision::VARCHAR)
  FROM TABLE(DATA_SOURCE(TYPE => 'STREAMING')));
```

This is a *pipe definition*; it does not send a single event and does not start a client. It also assumes a compatible target table, approved role, supported region and validated schema contract. Real schemas must specify whether malformed values are rejected, quarantined or represented as NULL. To reduce accidental data loss, reserve business validation for a downstream controlled step when appropriate.

Preflight before a paid integration test
Record cloud/region/edition, account identifier (not credential), selected SDK or REST version, table types, privilege grants, client endpoint, pipe mode, channel model, event logging, expected ingest volume and maximum bill. Confirm target behavior for schema mismatch, retries, and non-supported data types. Set a stop condition and a cleanup plan.

A companion repository in this book is a research reference, not a tested deployment. The linked Iceberg example can help readers understand where streaming intersects open table formats, but copying it into production without checking licenses and version constraints would violate the book's own editorial policy.

Project reference: P069: Streaming to Iceberg reference. *Not independently executed; rights review pending.*

7.5 SDK-based and API-based streaming workflows

7.5.1 SDK setup and integration responsibilities

A useful ingestion client is more than an API loop. It initializes a secure session, discovers pipe/channel capability, accepts source events, serializes a supported type representation, sends bounded batches, tracks per-record outcomes and preserves a restartable source checkpoint. For a managed broker, the source partition and offset form a natural ordering domain. For a custom REST source, the producer must define an equivalent stable sequence or cursor.

Reference architecture contract
Source adapter owns source reading and backpressure. **Mapper** owns schema-version selection and type conversion. **Ingest client** owns supported Snowflake API calls. **Checkpoint manager** owns source partition and last safely committed position. **Quarantine** owns invalid inputs with reason codes. **Observability** owns counts, retry age, clock deltas and costs.

Illustrative pseudocode (not an SDK API):

```
for source_partition in partitions:
    channel = open_supported_channel(source_partition)
    start = committed_offset(channel) + 1
    for event in broker.read_from(start):
        validate_envelope(event)
        submit_to_ingestion(channel, event)
        wait_for_confirmed_commit(channel, event.offset)
        broker.checkpoint(source_partition, event.offset)
```

Do not mistake those intentionally abstract function names for calls in the Snowflake Python SDK. Real SDKs may commit asynchronously, batch internally or expose separate methods for channel status. The point is the ordering of *source advancement* relative to *verified destination commit*. A source checkpoint written too early can lose data; one written too late can cause repeats. Implement a deterministic replay policy rather than demanding that a network never fails.

Security boundary. Prefer organization-approved short-lived identity mechanisms and supported Snowflake authentication. Secrets never appear in code examples, notebooks, printed screenshots, logs or the repository history. Restrict the ingest principal to its dedicated target and apply separate privileges for metering and administration.

Project reference: P071: Python ingestion interface reference. *Not independently executed; rights review pending.*

7.5.2 Authentication, channels, and offsets

A *channel* allows a client to relate a stream of submissions to durable progress. Named-channel workflows in the current high-performance design support offset-token tracking, while elastic-channel endpoints have at-least-once delivery behavior and avoid some lifecycle responsibilities. The ingestion guarantee describes a component boundary; exactly-once *business totals* still depend on event identity and downstream logic, especially across source corrections and multiple partitions.

An offset token belongs to the source application's chosen position scheme. Treat it as an opaque marker unless your source defines comparable numeric offsets; do not sort arbitrary offset strings lexicographically to infer broker position. Persist source partition, token, schema-version mapping and client identity together in a checkpoint ledger. When opening a channel after restart, reconcile destination committed status and source checkpoint before producing a new sequence.

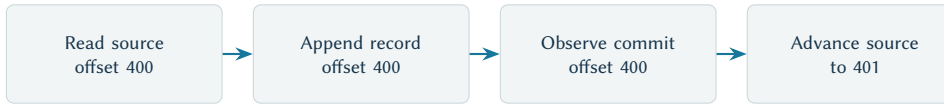


Figure 7.4. Durable acknowledgement must precede safe source advancement.

Correctness invariant
For a single source partition, no source position greater than the last acknowledged committed position is permanently discarded from retry storage. The application may retry an uncertain submission; that must not silently multiply silver-layer records.

To test offsets, inject a failure at three boundaries: before submission, after submission but before acknowledgement, and after acknowledgement but before the upstream checkpoint. The first is a straightforward retry; the second and third test idempotence. Capture the exact destination and source states so a future maintainer can reproduce the outcome rather than rely on folklore.

Project reference: P072: Java ingestion reference. *Not independently executed; rights review pending.*

7.5.3 Errors, disconnections, and reconnect design

The client needs two failure classes. *Retryable faults* include an interrupted network call, temporary throttling or unavailability. *Non-retryable contract faults* include malformed or forbidden payloads, an invalid schema version or denied permissions until configuration changes. Infinite retries on a permanent error hide the cause and block useful work; immediate discard after a temporary timeout risks loss.

A sensible strategy applies bounded exponential backoff with jitter, a retry budget, explicit quarantine for poison records, and preserved source offsets. Enforce maximum queue memory: if the destination cannot keep up, pause or throttle the source rather than letting a process grow indefinitely. Log at least producer partition, source token, pipeline step, reason code, attempts, first-failure time and eventual outcome, without logging credentials or sensitive payloads.

Listing 6: Illustrative retry decision, not a Snowflake SDK call.

```
if error.kind == 'TRANSIENT' and retries < limit:
    delay = min(max_wait, base_wait * 2 ** retries)
    schedule_retry(event_id, delay)
elif error.kind == 'CONTRACT':
    quarantine(event_id, error.reason)
else:
    stop_partition_and_alert(event_id)
```

Runbook: the channel reports an error
(1) Freeze or slow upstream offsets. (2) Inspect channel error/status and event log without exposing payload secrets. (3) Find last committed offset. (4) Distinguish rejected rows from unacknowledged rows. (5) Reopen with the documented SDK recovery method. (6) Replay bounded source positions. (7) Compare raw and silver counts. (8) Record final incident timeline.

In the simulator, source offset 8 fails once and then succeeds. We record *two attempts but one committed event.* That simple test is stronger than a screenshot showing a green connection. In a cloud exercise, preserve the client version, run identifier and actual channel-status output before asserting the system recovered. **Check.** What happens if one poison event blocks a strictly ordered source partition? Propose a policy that preserves its identity and error while making later healthy events available, without lying about contiguous commit progress.

7.6 Event ordering, duplicates, and late-arriving data

7.6.1 Arrival order is not event order

Source systems produce *event time*, transport imposes an arrival sequence, and the ingest client may commit asynchronously. These orderings are different. A sensor reading for 09:55 may arrive after 10:00 data because a device was offline, a retry waited, or a cloud region recovered. Sorting rows by ingest timestamp would make yesterday's correction appear newer than today's physical observation. Instead retain event time, original source partition/offset, ingest time, producer revision and an application-defined identity.

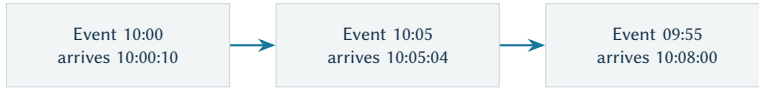


Figure 7.5. Physical observations can arrive out of event-time order.

7.6.2 Duplicates and idempotent analytical writes

A repeated network acknowledgement, a retried event or two overlapping source extracts may cause an input to appear more than once. Define the deduplication key by *business identity*: for our case `(provider, technology, event_utc)` together with a revision ordering. Raw bronze may intentionally retain duplicate deliveries; silver must represent the latest approved revision once. A source event ID alone is insufficient when a legitimate correction uses a new message ID.

Listing 7: Select approved latest revisions at the analytical grain.

```
SELECT * EXCLUDE rn FROM (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY provider, technology, event_utc
    ORDER BY revision DESC, received_utc DESC
  ) rn FROM energy_events_bronze
  WHERE energy_mwh >= 0
) WHERE rn = 1;
```

Tie-breaking policy matters. If two events carry the same business key and revision but different values, select neither arbitrarily: quarantine the conflict or escalate based on the publisher contract.

Project reference: P073: Kafka streaming demonstration. *Not independently executed; rights review pending.*

7.6.3 Late corrections, watermarks, and honest freshness

A *watermark* is a declared bound on how far event-time processing is expected to lag. It is not simply the maximum observed timestamp. Consider a five-minute aggregation that waits up to two additional minutes for stragglers. If a reading for 10:00 arrives at 10:09, the system needs an explicit correction policy: revise a historical aggregate, publish an adjustment record, or reject it after a documented business cutoff. Every choice affects downstream trust.

For Finland energy reporting, retain UTC timestamps in the transport and fact grain; convert to `Europe/Helsinki` only for display or reporting periods with daylight-saving awareness. A local hour may be skipped or repeated around the daylight-saving transition. A source event with a local clock string but no offset is ambiguous on such dates and needs publisher rules, not a guessed conversion.

Three separate notions of late
Transport late: it was published promptly but queued. *Publisher late:* the source only sent it after the measurement. *Correction late:* the publisher issues a higher revision after downstream consumers had finalized a value. A single ingestion-lag number does not diagnose all three.

Listing 8: Synthetic event age (seconds), not a service benchmark.

```
event_time = 0       # baseline UTC epoch in a toy clock
visible_time = 485   # illustrative late observation
age_seconds = visible_time - event_time
assert age_seconds > 300
```

Our lab retains one synthetic 09:55 observation arriving after 10:00 data. That late event increases the maximum event-to-bronze-visibility age even though it might have been ingested promptly after publication. For KPI reporting, measure the lag from the appropriate contractual clock and report late-data revisions separately.

Production check. Define a publishable hourly total when 99.9% of expected interval keys arrive by minute 4, but a late correction is permitted for 48 hours. Record whether the displayed result is preliminary or finalized, and ensure analysts can identify version changes. *Fast and correct* can mean versioned, not frozen.

Project reference: P074: CDC plus Snowpipe pattern. *Not independently executed; rights review pending.*

7.7 Schema changes and failure recovery

7.7.1 Versioned payload mapping

A new optional quality flag may be backwards compatible, while changing `energy_mwh` into `energy_kwh` is a semantic breaking change despite valid JSON. Accept only documented schema versions, preserve the untouched message and keep conversion rules in a tested mapper. Route unknown versions to a quarantine table until reviewed. Chapter 6's missing-versus-null checks remain essential in a continuous feed, where a bad schema can arrive thousands of times before somebody notices.

7.7.2 Interrupted ingestion and state recovery

A restart needs a known good source cursor and an independent destination commit record. Restore client identity, inspect named-channel status if supported, reconcile last committed positions, and replay the overlap needed to cover uncertain acknowledgement. Never advance the upstream cursor based solely on a client-side "request sent" message. For multi-partition sources, track independent progress; a single global counter can mask one stalled partition.

7.7.3 Replay, quarantine, and safe cutover

Backfills can interact with live traffic. Choose a bounded event range, preserve original event IDs and revision numbers, and make the silver merge idempotent. Quarantine records require an explicit disposition: reprocess after a mapper fix, replace from an authoritative snapshot, or mark irrecoverable with a signed reason. Do not silently drop them to make the dashboard healthy.

Failure injection matrix
Disconnect before send: retry from source. **Commit uncertain:** verify channel progress, then replay safely. **Schema mismatch:** quarantine with version and field. **Duplicate delivery:** retain bronze provenance but prevent duplicate silver totals. **Late revision:** update the relevant analytical interval with an audit trail.

Check. A live feed begins publishing `unit='kWh'` without incrementing its schema version. Design the alarm, containment, replay and backfill steps that prevent an unnoticed thousandfold reporting error.

Project reference: P075: Stream data generator. *Not independently executed; rights review pending.*

Project reference: P076: Multi-service streaming reference. *Not independently executed; rights review pending.*

7.8 Openflow and connector-based ingestion overview

7.8.1 When connectors are useful

A purpose-built connector can own source polling, authentication, snapshot/CDC semantics, schema discovery and transformations that would otherwise take many lines of custom code. Its value is a maintained operational contract, not a magical guarantee that every row is correct. Evaluate connector lag, replay support, source permissions, observability, dependency lifecycle and change handling just as you would a bespoke consumer.

7.8.2 Openflow deployment boundaries

Snowflake Openflow provides managed integration workflows with deployment options including Snowpark Container Services-hosted runtimes and bring-your-own-cloud arrangements. A deployment contains runtimes, and a runtime executes one or more integration flows according to supported configuration. Do not assume the same integration features, preview status or runtime controls across deployment generations and cloud regions. The currently documented platform includes generation-specific limitations and some preview configuration features.

7.8.3 Infrastructure, observability, and hidden costs

Openflow can add running control-plane/compute-pool costs, per-runtime compute, ingestion credits, logging/telemetry and external transfer or CDC warehouse charges. An idle-looking flow may still incur compute-pool costs if its management resources remain running. Separate an *integration demo* from the cheapest way to ingest a small periodic dataset. Before choosing Openflow, compare a minimal direct Snowpipe integration on identical acceptance criteria.

Connector review questions
Does it support our database and precise CDC version? Can it recover after a multi-hour outage without snapshots duplicating rows? Who owns schema evolution? Which components keep billing when paused? What credentials are required? Can we export complete source offsets and raw events if the service is replaced?

Example choice. A PostgreSQL transactional source with ongoing CDC and multiple tables may justify an Openflow connector. A single weekly public energy CSV probably does not. Prefer the design with the least operational burden that meets measurable freshness, security and traceability requirements.

Project reference: P021: Openflow on SPCS reference. *Not independently executed; rights review pending.*

Project reference: P077: Openflow structured pipeline. *Not independently executed; rights review pending.*

Project reference: P078: Openflow PostgreSQL CDC. *Not independently executed; rights review pending.*

7.9 Latency, monitoring, throughput, and cost

7.9.1 Measure the full event-to-table path

Measure pipeline stages separately: source observation, source publish, broker receipt, client read, Snowflake query visibility and silver readiness. Set clocks to UTC; record their precision and synchronization error. A message that is three minutes old at source publication cannot be fixed solely by changing the ingest product. Conversely, if publish-to-visible grows under pressure, inspect queue depth and client backlog before changing SQL transformations.

Listing 9: Freshness query against an approved raw table.

```
SELECT technology,
 DATEDIFF('second', event_utc, received_utc)
   AS source_age_seconds
FROM energy_telemetry_raw
WHERE event_utc IS NOT NULL;
```

Measurement caveat: `received_utc` is a stored column, not proof of the query-visible timestamp. Use a separately recorded acknowledgment, controlled probe query, or documented channel metric to bound ingest visibility in a real benchmark. Instrument silver completion independently. For p95/p99, report sample count and the observation window, not only a representative screenshot.

7.9.2 Throughput, backlog, and backpressure

Define upstream offered rate λ and sustained downstream service rate μ. When $\lambda > \mu$ for long enough, backlog grows regardless of optimistic average latency claims. Little's law relates long-run mean items-in-system L, throughput λ, and residence time W under stable conditions: $L = \lambda W$. Use it diagnostically, not as an excuse to assume stationarity during an outage.

Alert example
Alert if the oldest uncommitted source offset ages beyond the freshness objective for three consecutive evaluation windows, or if one partition stops advancing while others continue. Join this to acceptance/quarantine rates; a silent reduction in accepted rows can make throughput look "fast" while dropping data.

Check. A client handles 500 messages/s but the source produces 650 messages/s for 10 minutes. What backlog accumulated *ignoring retries*? Why is an average ingest latency measured before the peak uninformative about recovery time?

7.9.3 Separating ingestion billing from pipeline economics

Compare costs using the account's actual service consumption table and a time-bounded meter export. File-based Snowpipe usage can be examined in `PIPE_USAGE_HISTORY` in the Snowflake Account Usage schema, which has reporting latency. High-performance Snowpipe Streaming is metered separately according to its documented throughput-oriented model. Traditional warehouses, connector runtimes and downstream transformations are separate billable resources. Resource monitors that stop user-managed warehouses do not, by themselves, cap every serverless ingestion or connector charge.

Listing 10: Auditable file-based Snowpipe meter query; requires privileges.

```
SELECT TO_DATE(start_time) AS usage_day,
       pipe_name,
       SUM(credits_used) AS credits_used,
       SUM(bytes_billed) AS bytes_billed
FROM snowflake.account_usage.pipe_usage_history
WHERE start_time >= DATEADD(day,-7,CURRENT_TIMESTAMP())
GROUP BY 1,2 ORDER BY 1,2;
```

This query reports service usage, not end-to-end energy-dashboard cost. Add storage, query warehouses, stream processing, orchestration, network and on-call cost before comparing architectures. Document reporting lags; do not describe incomplete last-hour metering as zero charges.

Five metrics to keep
(1) Accepted input events/s; (2) p95 event-to-bronze age; (3) p95 event-to-silver readiness; (4) partition offset lag and oldest retry age; (5) ingestion *and* downstream credits per million approved observations. Always report quarantines separately.

Reading trail and status. Snowflake documentation checkpoint: 22 October 2026. Full links appear in the companion README and the references here; functionality, prices and edition limitations may change. Primary references: Snowpipe auto-ingest; CREATE PIPE; streaming architectures; named-channel recovery; Openflow costs; and file Snowpipe costs. No live cloud ingest or cost benchmark was performed.

Project reference: P079: Event streaming presentation reference. *Not independently executed; rights review pending.*

Project reference: P080: Community end-to-end streaming demo. *Not independently executed; rights review pending.*

7.L Guided lab: a failing energy telemetry feed

7.L.1 Inputs, environment, permissions, and cost

The companion's `streaming_simulator.py` runs with Python 3.10+ and the standard library only. It models one ordered source partition over nine synthetic events. An exact duplicate, a higher-revision correction, a late arrival, a negative measurement and a retry after transient failure are deliberately included. No network access, Snowflake account, API key or credits are required for the offline experiment.

The optional SQL files are *integration templates*, not executed database scripts. Only run cloud components after an administrator approves a disposable schema, a narrowly scoped role, storage integration, region and cost limit. A file-based Snowpipe definition needs an actual authorized external stage; a high-performance streaming pipe needs a matching current SDK/REST client. Never publish secrets or assume a commented pipe template provides an event queue.

7.L.2 Execute and inspect the deterministic experiment

Listing 11: From the extracted Chapter 7 lab folder.

```
python streaming_simulator.py --out output
python -m unittest -v test_streaming_simulator.py
```

Inspect `output/bronze.csv` and `output/silver.csv`. Bronze preserves eight accepted deliveries, including the duplicate and the older wind revision. Silver selects one approved revision for each (provider, technology, UTC interval) key. The negative reading goes to `output/quarantine.json`; the failure at source offset 8 is recorded in `attempts.json` before replay succeeds. The final source offset is committed only after a successful attempt.

Deterministic simulator output (offline)	Result
Source records / delivery attempts	9 / 10
Accepted bronze / quarantined	8 / 1
Unique latest-revision silver facts	6
Silver total, MWh	66.000
Median / maximum event-to-visible age, seconds	27 / 485
Last safely committed synthetic offset	8

Table 7.3. Measured from synthetic fixture clocks, not Snowflake.

Failure interpretation. The large maximum reflects a deliberately late observation, not necessarily a slow ingest service. Reporting it as Snowflake's p99 latency would be a serious measurement error. Distinguish accepted source delivery from published business readiness and document the assumed clock domains.

7.L.3 Assertions, diagnosis, replay, and cleanup

The eight offline tests check row accounting; higher-revision precedence; total MWh; quarantine; retry/commit order; late-event detection; deterministic replay; and the no-failure baseline. The strongest invariants are: accepted bronze plus quarantined equals unique source messages; silver contains at most one approved latest revision per business key; replay does not increase the business total; and the committed source offset never skips an unprocessed failure.

If the count differs, inspect `attempts.json`. If energy total increases on a second run, inspect business-key deduplication and revision precedence. If freshness is negative, validate event and visibility clocks before treating the message as fast. **Cleanup:** remove the local `output/` directory; the lab creates no cloud resources. Cloud teardown must be approved separately after actual integration testing.

7.L.4 Practice, architecture decision, and assessment

Foundation exercise. Add a second duplicate delivery and assert that bronze count increases while silver count and total remain stable. **Practitioner exercise.** Add a late higher-revision correction for hydro and prove that the final energy total changes exactly once. **Production decision.** Design a rollout from hourly file loads to row-based streaming for an energy alarm. Specify source partitions, channel mode, credentials, safety limit, backpressure, clock measurements, reconciliation and rollback. Decide whether the extra speed is worth its operational obligations.

Assessment rubric (10 points)
Architecture selection with measurable freshness (2); explicit source/target acknowledgment invariant (2); duplicate and revision-correction handling (2); quarantine and replay evidence (2); cost, privacy and cleanup plan (2). An architectural choice without source-offset safety cannot receive full marks.

Key takeaways. "Continuous" describes delivery, not guaranteed freshness; exactly-once semantics have a specified boundary; event-time ordering is distinct from arrival order; poison records need a named disposition; and costs include service-side compute, cloud plumbing and operating responsibility. Chapter 8 will consume this trusted bronze feed through Snowflake Streams, Tasks and Dynamic Tables, keeping ingestion and subsequent change processing as separate concerns.

Publishing verification gate.
At print approval, confirm all cited APIs, region/edition constraints, ingest and channel costs, repository licenses and client versions; run selected SQL in a bounded Snowflake test account; capture execution IDs, date, actual usage and cleanup proof. The present chapter does *not* claim those cloud tests were completed.